# Multimedia Signal Processing - 2nd module

Prof. Marco Tagliasacchi

July, 1 2010

## 1 Question - 10 pts

Consider a gray-scale image with 4 grey levels $L = 0, 1, 2, 3$. The following statistics can be estimated collecting data from an image collection:

$$p(0) = \Pr(l = 0) = 0.25$$
$$p(1) = \Pr(l = 1) = 0.25$$
$$p(2) = \Pr(l = 2) = 0.35$$
$$p(3) = \Pr(l = 3) = 0.15$$

- Compute the entropy of the source.
- Design a Huffman code using block sizes $N = 1, 2$.
- In each case, compute the average code length $L_N$ and generate the compressed bitstream corresponding to the following sequence of pixel values, read in raster scan order: $[1, 1, 2, 0]$.

Inter-pixel correlation is described by the following statistics

$$P = \begin{bmatrix} 0.2 & 0.05 & 0 & 0 \\ 0.05 & 0.2 & 0 & 0 \\ 0 & 0 & 0.3 & 0.05 \\ 0 & 0 & 0.05 & 0.1 \end{bmatrix} \quad (1)$$

where $p_{ij}$ denotes the probability $\Pr(l_1 = i, l_2 = j)$, given two consecutive pixels $l_1$ and $l_2$ in raster scan order.

- Show that pixel values of consecutive pixels are statistically dependent
- Compute the joint entropy
- Design a Huffman code using block size $N = 2$ and compute the average code length.

### Solution

- The entropy of the source is

$$H(X) = -0.25 \log_2 0.25 - 0.25 \log_2 0.25 - 0.35 \log_2 0.35 - 0.15 \log_2 0.15 = 1.94\text{bps} \quad (2)$$

- A possible Huffman code for $N = 1$ is

| symbol | codeword | length |
|---|---|---|
| 0 | 01 | 2 |
| 1 | 10 | 2 |
| 2 | 11 | 2 |
| 3 | 00 | 2 |

The average code length is $L_1 = 2$ bps.

In order to construct a Huffman code for $N = 2$, we need to compute the probabilities of supersymbols. That is

$$
\begin{aligned}
p(a) = p(0,0) &= \Pr(l=0)\Pr(l=0) = 0.25 \cdot 0.25 = 0.0625 \\
p(b) = p(0,1) &= \Pr(l=0)\Pr(l=1) = 0.25 \cdot 0.25 = 0.0625 \\
p(c) = p(0,2) &= \Pr(l=0)\Pr(l=2) = 0.25 \cdot 0.35 = 0.0875 \\
p(d) = p(0,3) &= \Pr(l=0)\Pr(l=3) = 0.25 \cdot 0.15 = 0.0375 \\
p(e) = p(1,0) &= \Pr(l=1)\Pr(l=0) = 0.25 \cdot 0.25 = 0.0625 \\
p(f) = p(1,1) &= \Pr(l=1)\Pr(l=1) = 0.25 \cdot 0.25 = 0.0625 \\
p(g) = p(1,2) &= \Pr(l=1)\Pr(l=2) = 0.25 \cdot 0.35 = 0.0875 \\
p(h) = p(1,3) &= \Pr(l=1)\Pr(l=3) = 0.25 \cdot 0.15 = 0.0375 \\
p(i) = p(2,0) &= \Pr(l=2)\Pr(l=0) = 0.35 \cdot 0.25 = 0.0875 \\
p(l) = p(2,1) &= \Pr(l=2)\Pr(l=1) = 0.35 \cdot 0.25 = 0.0875 \\
p(m) = p(2,2) &= \Pr(l=2)\Pr(l=2) = 0.35 \cdot 0.35 = 0.1225 \\
p(n) = p(2,3) &= \Pr(l=2)\Pr(l=3) = 0.35 \cdot 0.15 = 0.0525 \\
p(o) = p(3,0) &= \Pr(l=3)\Pr(l=0) = 0.15 \cdot 0.25 = 0.0375 \\
p(p) = p(3,1) &= \Pr(l=3)\Pr(l=1) = 0.15 \cdot 0.25 = 0.0375 \\
p(q) = p(3,2) &= \Pr(l=3)\Pr(l=2) = 0.15 \cdot 0.35 = 0.0525 \\
p(r) = p(3,3) &= \Pr(l=3)\Pr(l=3) = 0.15 \cdot 0.15 = 0.0225
\end{aligned}
\tag{3}
$$

A possible Huffman code for $N = 2$ is

| symbol | codeword | length |
|---|---|---|
| 0,0 | 1100 | 4 |
| 0,1 | 1101 | 4 |
| 0,2 | 1001 | 4 |
| 0,3 | 10000 | 5 |
| 1,0 | 1110 | 4 |
| 1,1 | 1111 | 4 |
| 1,2 | 000 | 3 |
| 1,3 | 10001 | 5 |
| 2,0 | 001 | 3 |
| 2,1 | 1010 | 4 |
| 2,2 | 011 | 3 |
| 2,3 | 0101 | 4 |
| 3,0 | 10110 | 5 |
| 3,1 | 01001 | 5 |
| 3,2 | 10111 | 5 |
| 3,3 | 01000 | 5 |

The average code length is $L_2 = 3.9275/2 = 1.9638$

- The sequence $[1, 1, 2, 0]$ is encoded as
  - 10101101
  - 1111001
- In order to show that pixel values are not statistically independent, it suffices to show that, for some $i, j$, $\Pr(l_1 = i, l_2 = j) \neq \Pr(l_1 = i) \cdot \Pr(l_2 = j)$. For example, $\Pr(l_1 = 0, l_2 = 0) = 0.2 \neq \Pr(l_1 = 0) \cdot \Pr(l_2 = 0) = 0.0625$.
- The joint entropy is

$$
H(X_1, X_2) = -0.2\log_2 0.2 - 0.05\log_2 0.05 - 0.05\log_2 0.05 - 0.2\log_2 0.2 - 0.3\log_2 0.3 - 0.05\log_2 0.05 - 0.3\log_2 0.3 = 2.6464 \tag{4}
$$

- | symbol | p | codeword | length |
  |---|---|---|---|
  | 0,0 | 0.2 | 00 | 2 |
  | 0,1 | 0.05 | 10100 | 5 |
  | 1,0 | 0.05 | 10101 | 5 |
  | 1,1 | 0.2 | 01 | 2 |
  | 2,2 | 0.3 | 11 | 2 |
  | 2,3 | 0.05 | 10110 | 5 |
  | 3,2 | 0.05 | 10111 | 5 |
  | 3,3 | 0.1 | 100 | 3 |

  The average code length is $\bar{L}_2 = 1.3500$. Exploiting the memory, the saving is $L_2 - \bar{L}_2 = 1.9638 - 1.3500 = 0.6138$ bps.

# 2 Question - 5 pts

Describe the use of short and long term predictors in speech coders.

## Solution

See slide set `MMSP_2.1_Speech coding`.

# 3 Questions - 5 pts (each answer can be either TRUE or FALSE)

1. Consider a quantization scheme based on compander-expander. Let $y = T(x)$ denote the nonlinear transform and $R_c(D)$ the corresponding rate-distortion function.
   - [T] [F] $T(x) = \int_{-\infty}^{+\infty} p(x)dx$
     FALSE. $T(x) = \int_{-\infty}^{+x} p(a)da$
   - [T] [F] $R_c(D) \geq R_{ECSQ}(D)$, for all $D \geq 0$
     TRUE. For a target distortion $D$, ECSQ is the optimal quantizer, i.e. the one that minimizes the number of bits needed to encode the source.
   - [T] [F] The output quantization symbols are uniformly distributed.
     TRUE.
2. Consider a perceptual audio coder
   - [T] [F] The analysis filter bank is the same used by the perceptual model
     FALSE. Typically different filter banks are used.
   - [T] [F] Noise and tonal maskers are characterized by different SMR
     TRUE. Noise is a stronger masker than tones.
   - [T] [F] The critical bandwidth decreases at higher frequencies
     FALSE. Critical bandwidth is constant at low frequency, and increases at higher frequencies.
   - [T] [F] The bitrate depends on the audio content to be encoded
     TRUE. (The question is actually a bit ambiguous). If rate-control is not enabled, for a target perceived quality the bitrate depends on the content.
3. Consider a hybrid DPCM/DCT video coding system
   - [T] [F] Decoding is computationally more complex than encoding
     FALSE. Encoding complexity dominates due to motion estimation.
   - [T] [F] Motion estimation is performed in the DCT domain
     FALSE. Motion estimation is performed in the pixel domain.
   - [T] [F] Temporal redundancy is addressed using DPCM
     TRUE. A closed-loop DPCM is used along the temporal direction.

# 4 Question - 12 pts

Consider the `rice` image (`I =imread('rice.png');`).
Implement a baseline version of the JPEG compression algorithm with the following simplifications:

- Luminance component only.
- No DPCM coding of DC coefficient.
- No entropy coding.

The quantization matrix is given by (also available in the attached file `Qjpeg.mat`):
Compute the rate-distortion curve for the `rice` image by varying the quantization parameter (plot PSNR vs. Q).
Now, consider corrupting the image in the following ways

$$\begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

- Adding white Gaussian noise with standard deviation $\sigma_z = 100$
- Adding a luminance offset

In both cases, compute the rate-distortion curve. Compare the curve with the original one and comment the results.

## Solution

```
clear all;
close all;
clc;

load Qjpeg.mat;

I = double(imread('rice.png'));
[M N] = size(I);
blocksize = 8;

%quality factor
Q = 1:15;

%output
Iout = NaN*ones(size(I));

TYPE = {'normal', 'noise', 'luminance'};
SIGMA = 100;
NOISE = SIGMA*randn(size(I));
OFFSET = round(max(max(NOISE)));

PSNR = NaN*ones(length(TYPE),length(Q));

for itype = 1:length(TYPE)

    type = TYPE(itype);
    switch(char(type))
        case 'normal'
            %do nothing
            Iin = I;
        case 'noise'
            %add noise
            Iin = I + NOISE;
        case 'luminance'
            %add luminance offset
            Iin = I + OFFSET;
    end

    for q = Q

        qjpeg = q*QJPEG;

        for ir=0:(M/blocksize)-1
            for ic=0:(M/blocksize)-1
                %extract 8x8 block
                Ib = Iin(blocksize*ir+1:blocksize*(ir+1),blocksize*ic+1:blocksize*(ic+1));

                %compute 2D dct
                Ibdct = dct2(Ib);

                %quantize dct coefficients
                Ibdct_q = qjpeg.*round(Ibdct./qjpeg);

                %compute inverse dct transforms
                Ib_q = idct2(Ibdct_q);

                %copy block in the compressed image
                Iout(blocksize*ir+1:blocksize*(ir+1),blocksize*ic+1:blocksize*(ic+1)) = Ib_q;
            end
        end

        Ierr = Iin-Iout;
        MSE = var(Ierr(:));
        PSNR(itype,q) = 10*log10((2^8-1)^2/MSE);
```

```
    end
end

plot(Q,PSNR);
legend('normal','noise','luminance offset');
xlabel('Quality factor');
ylabel('PSNR [db]');
```